

Prior to `add_disk()`, the various fields of struct `gendisk` need to be initialised, either directly or using various macros/functions like `set_capacity()`. `major`, `first_minor`, `fops`, `queue`, `disk_name` are the minimal fields to be initialised directly. And even before the initialisation of these fields, the struct `gendisk` needs to be allocated, using the function given below:

```
struct gendisk *alloc_disk(int minors);
```

Here, `minors` is the total number of partitions supported for this disk. And the corresponding inverse function would be:

```
void put_disk(struct gendisk *disk);
```

All these are prototyped in `<linux/genhd.h>`.

Request queue and the request function

The request queue also needs to be initialised and set up into the struct `gendisk`, before `add_disk()`. The request queue is initialised by calling:

```
struct request_queue *blk_init_queue(request_fn_proc *,
spinlock_t *);
```

We provide the request-processing function and the initialised concurrency protection spin-lock as parameters. The corresponding queue clean-up function is given below:

```
void blk_cleanup_queue(struct request_queue *);
```

The request (processing) function should be defined with the following prototype:

```
void request_fn(struct request_queue *q);
```

It should be coded to fetch a request from its parameter `q`, for instance, by using the following:

```
struct request *blk_fetch_request(struct request_queue *q);
```

Then it should either process it, or initiate processing. Whatever it does should be non-blocking, as this request function is called from a non-process context, and also after taking the queue's spin-lock. Moreover, only functions not releasing or taking the queue's spin-lock should be used within the request function.

A typical example of request processing, as demonstrated by the function `rb_request()` in `ram_block.c` is given below:

```
while ((req = blk_fetch_request(q)) != NULL) /* Fetching a
request */
{
```

```
/* Processing the request: the actual data transfer */
ret = rb_transfer(req); /* Our custom function */
/* Informing that the request has been processed with
return of ret */
__blk_end_request_all(req, ret);
}
```

Requests and their processing

Our key function is `rb_transfer()`, which parses a struct request and accordingly does the actual data transfer. The struct request primarily contains the direction of data transfer, the starting sector for the data transfer, the total number of sectors for the data transfer, and the scatter-gather buffer for the data transfer. The various macros to extract these from the struct request are as follows:

```
rq_data_dir(req); /* Operation type: 0 - read from device;
otherwise - write to device */
blk_req_pos(req); /* Starting sector to process */
blk_req_sectors(req); /* Total sectors to process */
rq_for_each_segment(bv, req, iter) /* Iterator to extract
individual buffers */
```

`rq_for_each_segment()` is the special one which iterates over the struct request (`req`) using `iter`, and extracting the individual buffer information into the struct `bio_vec` (`bv`; basic input/output vector) on each iteration. And then, on each extraction, the appropriate data transfer is done, based on the operation type, invoking one of the following APIs from `ram_device.c`:

```
void ramdevice_write(sector_t sector_off, u8 *buffer, unsigned
int sectors);
void ramdevice_read(sector_t sector_off, u8 *buffer, unsigned int
sectors);
```

Check out the complete code of `rb_transfer()` in `ram_block.c`.

Summing up

With that, we have actually learnt the beautiful block drivers by traversing through the design of a hard disk and playing around with partitioning, formatting and various other raw operations on a hard disk. Thanks for patiently listening. Now, the session is open for questions. Or, you may send across your queries to email@sarika-pugs.com. 

By: Anil Kumar Pugalia

The author is a freelance trainer in Linux internals, Linux device drivers, embedded Linux and related topics. Prior to this, he was at Intel and Nvidia. He has been working with Linux since 1994. A gold medalist from the Indian Institute of Science, Linux and knowledge sharing are two of his many passions. Creating and playing with open source hardware is one of his hobbies. Learn more about him at <http://profession.sarika-pugs.com/>, including information about his open source hardware freak-outs and his various Linux related training sessions and workshops. He can be reached at email@sarika-pugs.com.